

5.4 Using Dependencies

Modern test frameworks provide the option of defining spies, stubs, and mocks, which are all also called test doubles. You can think of this as a function whose calls you can check (using the *spy*), whose behavior you can predefine (using the *stub*), or whose interaction rules you can define and then check whether they have been adhered to at the end of a test (using the *mock*). Spies and stubs in particular are very useful when it comes to formulating unit tests. If you use callback functions, such as event handlers, you can't avoid using spy functions.

In the following sections, we'll focus on the use of stubs to write unit tests for functions that have additional dependencies. In a unit test, you focus on a specific unit of your source code, usually a function or method. As soon as this works together with other parts of your application, the situation becomes more complex. For one thing, the test can no longer fail because of just one function, and for another, you potentially have to cover multiple flow paths, which means that there is a larger number of tests to be performed.

To make the whole thing more concrete, we'll assume a TypeScript class named `OrderService` in the following example. This has a `placeOrder` method that enables you to save an order. When saving, your application enriches the order with additional information and uses another class—the `OrderRepository` class—to persist the order. The greatly simplified source code of these two classes can be found in the following code example:

```
import { randomUUID } from "node:crypto";

type Item = {
  id: string;
  name: string;
  price: number;
```

```

    quantity: number;
  };
  type Order = {
    id: string;
    items: Item[];
    total: number;
    status: "new" | "pending" | "completed" |
    "cancelled";
    createdAt?: Date;
  };

  export class OrderRepository {
    async save(order: Order): Promise<Order> {
      return {
        ...order,
        id: randomUUID(),
        createdAt: new Date(),
      };
    }
  }

  export class OrderService {
    constructor(private orderRepository:
    OrderRepository) {}

    placeOrder(order: Order) {
      const total = order.items.reduce((acc, item) => {
        return acc + item.price * item.quantity;
      }, 0);

      const orderRecord: Order = {
        ...order,
        total,
        status: "pending",
      };

      return this.orderRepository.save(orderRecord);
    }
  }
}

```

When it gets instantiated, `OrderService` receives an instance of the `OrderRepository` class. If you call the `placeOrder` method with an `Order` object,

the method enriches this object with the total amount of the ordered parts and sets the status to pending. The method then calls the save method of the OrderRepository object. This method in turn assigns a unique ID to the order and sets the creation date.

The AI tool of our choice—in this case, ChatGPT—should generate a test for the placeOrder method and ensure that the save method of the OrderRepository isn't called, but rather is replaced by a stub method.

Prompt: Write a Vitest test for the placeOrder method, and replace the save method of the OrderRepository with a stub.

[code of the example]

Here's how you can write a test for the placeOrder method in your OrderService class using Vitest. In this test, we'll replace the save method of the OrderRepository with a stub.

First, ensure that you have Vitest installed and set up in your project. Then, create a test file, for example, OrderService.test.ts, and add the following code:

```
import { describe, it, expect, vi } from 'vitest';
import {
  OrderService, OrderRepository, Order, Item
} from './path-to-your-order-service';

describe('OrderService', () => {
  it('should place an order and calculate the total',
    async () => {
      // Create a stub for the OrderRepository save
method
      const saveStub = vi.fn().mockResolvedValue({
        id: 'stubbed-id',
        items: [],
        total: 100,
        status: 'pending',
        createdAt: new Date(),
      });
```

```

    // Create a mock OrderRepository
    const orderRepository = new OrderRepository();
    orderRepository.save = saveStub;
    // Instantiate the OrderService with the mock
    // OrderRepository
    const orderService = new
OrderService(orderRepository);

    // Define a sample order
    const order: Order = {
      id: '',
      items: [
        { id: '1', name: 'item1', price: 10, quantity:
5 },
        { id: '2', name: 'item2', price: 20, quantity:
2 },
      ],
      total: 0,
      status: 'new',
    };

    // Place the order
    const result = await
orderService.placeOrder(order);

    // Assertions
    expect(saveStub).toHaveBeenCalledTimes();
    expect(saveStub).toHaveBeenCalledTimes(
      expect.objectContaining({
        total: 90,
        status: 'pending',
      }));
    expect(result).toEqual(expect.objectContaining({
      id: 'stubbed-id',
      total: 90,
      status: 'pending',
    }));
  });
});
});

```

[explanation of the test]

The test first prepares the environment as required by creating a stub for the save method. It then creates a new `OrderRepository`, inserts the stub there, and generates a new instance of `OrderService`. The most important part of the test is the call of the `placeOrder` method. To do this, the test defines an order object that represents the order and passes it when the method is called. The test concludes with the following checks:

- Has the `saveStub` been called exactly once?
- Was the `saveStub` called with the correct object?
- Is the return value of the `placeOrder` method correct?

The test looks good so far, and you can integrate and execute it directly in your application. However, you'll then see a significant problem because the test fails when checking the return value of the `placeOrder` method. The reason for this is that the stub function returns the value `100`, for example, and not `90` as required. The `items` and `createdAt` properties are also missing.

You can solve this problem by copying the error message to the next prompt. In this case, ChatGPT revises the answer and provides you with a functioning test.

Prompt: I got this error, please fix the test:

```
AssertionError: expected { id: 'stubbed-id',
                           items: [], ...(3) }
                           to deeply equal ObjectContaining{...}
- Expected
+ Received

- ObjectContaining {
+ Object {
+   "createdAt": 2024-07-21T14:18:23.915Z,
+   "id": "stubbed-id",
+   "items": Array [],
+   "status": "pending",
-   "total": 90,
+   "total": 100,
+ }
> src/order.service.test.ts:49:20
    47 |         })
```

```

48 |     );
49 |     expect(result).toEqual(
    |                             ^
50 |         expect.objectContaining({
51 |             id: "stubbed-id"

```

The error indicates a mismatch between the expected and received objects in the assertion. Specifically, the total value and createdAt field are different. Let's update the test to correctly handle these discrepancies. Here is the updated test code:

```

import { describe, it, expect, vi } from 'vitest';
import {
    OrderService, OrderRepository, Order, Item
} from './path-to-your-order-service';
describe('OrderService', () => {
    it('should place an order and calculate the total',
        async () => {
            // Create a stub for the OrderRepository save
method
            const saveStub = vi.fn()
                .mockImplementation(async (order: Order) => ({
                    ...order,
                    id: 'stubbed-id',
                    createdAt: new Date(),
                }));

            // Create a mock OrderRepository
            const orderRepository = new OrderRepository();
            orderRepository.save = saveStub;

            // Instantiate the OrderService with the mock
OrderRepository
            const orderService = new
OrderService(orderRepository);

            // Define a sample order
            const order: Order = {
                id: '',
                items: [
                    { id: '1', name: 'item1', price: 10, quantity:
5 },

```

```

        { id: '2', name: 'item2', price: 20, quantity:
2 },
    ],
    total: 0,
    status: 'new',
  };

  // Place the order
  const result = await
orderService.placeOrder(order);

  // Assertions
  expect(saveStub).toHaveBeenCalledTimes();
  expect(saveStub).toHaveBeenCalledWith(expect
    .objectContaining({
      total: 90,
      status: 'pending',
    }));

  expect(result.id).toBe('stubbed-id');
  expect(result.total).toBe(90);
  expect(result.status).toBe('pending');
  expect(result.items).toEqual(order.items);
  expect(result.createdAt).toBeInstanceOf(Date);
});
});

```

In this updated test:

1. The `saveStub` now uses `mockImplementation` to return the order object with the `id` and `createdAt` fields added, simulating the actual save operation more closely.
2. The assertions are broken down to individually check each field, making the test clearer and avoiding deep equality checks that might fail due to differences in other fields (like `createdAt`).

ChatGPT has tackled precisely the two critical points when adapting the source code. The stub implementation is now based on the transferred object and only adds two properties. When checking the return value, the test now proceeds property by property so that no more errors can occur here

either. If you integrate this test into your environment, it will run without an error message.